



Deliverable 3 of 3

Technical Documentation

RAG-Based Customer Support Assistant with LangGraph and HITL

Project:	RAG-Based Customer Support Assistant
Prepared By:	Shravan Shidruk
Institute:	Innomatics Research Labs
Tech Stack:	Python, LangChain, LangGraph, ChromaDB, Ollama, Groq
Date:	April 2026

Table of Contents

1. Introduction

- 1.1 What is RAG
- 1.2 Why RAG is Needed
- 1.3 Use Case Overview

2. System Architecture Explanation

3. Design Decisions

4. Workflow Explanation

5. Conditional Logic

6. HITL Implementation

7. Challenges and Trade-offs

8. Testing Strategy

9. Future Enhancements

1. Introduction

1.1 What is RAG

Retrieval-Augmented Generation (RAG) is an AI architecture that enhances a large language model (LLM) by providing it with dynamically retrieved, context-specific information at inference time. Rather than relying solely on knowledge baked into the model's weights during training, a RAG system first searches an external knowledge source (typically a vector database) for passages that are semantically relevant to the user's query, then injects those passages into the LLM's prompt as context before generating an answer.

This two-step retrieval-then-generation pattern was introduced in the 2020 paper 'Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks' by Lewis et al. (Facebook AI Research) and has since become the dominant architecture for building grounded, factual AI assistants.

1.2 Why RAG is Needed

Pure LLMs have several limitations that make them unsuitable for production customer support systems without augmentation. First, LLMs have a knowledge cutoff date; they cannot answer questions about information created after training. Second, LLMs hallucinate: they generate plausible-sounding but factually incorrect statements when they lack knowledge. Third, training or fine-tuning an LLM on proprietary data is expensive and slow to update.

RAG directly addresses all three problems. The vector store is updated simply by re-ingesting the knowledge base, with no model retraining required. The LLM prompt explicitly instructs the model to answer only from the provided context, which dramatically reduces hallucination. And because the retrieval step is based on semantic similarity rather than keyword matching, the system finds relevant information even when the user's phrasing differs from the document's wording.

1.3 Use Case Overview

This project implements a customer support assistant for NovaTech, a fictional technology company. The assistant is backed by a PDF knowledge base (`data/knowledge_base.pdf`) that contains product documentation, policies, and FAQs. Users interact with the system via a command-line interface. The system answers questions autonomously where the knowledge base contains sufficient information, and escalates to a human operator for refund requests, payment issues, or queries where the retrieved context is insufficient to generate a confident answer.

2. System Architecture Explanation

The architecture consists of two temporally separated pipelines: the ingestion pipeline, which runs offline to populate the vector store, and the query pipeline, which runs online for every user interaction.

2.1 Ingestion Pipeline

The ingestion pipeline is triggered by running `main.py` for the first time (or when `setup_pipeline()` is called). `PyPDFLoader` reads the PDF page by page, producing `LangChain Document` objects. The chunker cleans each page's text to remove PDF extraction artifacts (excess whitespace, pipe characters from tables) and then splits the text using `RecursiveCharacterTextSplitter`. The splitting strategy prioritizes paragraph boundaries, then line breaks, then sentence endings, then word boundaries, ensuring chunks are as semantically coherent as possible within the 700-character limit. The 150-character overlap between adjacent chunks acts as a sliding window, reducing the chance that a relevant sentence is split across a chunk boundary without representation in either chunk.

`OllamaEmbeddings` generates a 768-dimensional vector for each chunk by calling the locally running `nomic-embed-text` model. These vectors are written to `ChromaDB`'s persistent store. `ChromaDB` uses an HNSW (Hierarchical Navigable Small World) index internally for approximate nearest-neighbor search, which scales to millions of vectors while maintaining sub-millisecond query times.

2.2 Query Pipeline and Component Interactions

When a user submits a query, `main.py` passes it to `graph.invoke()`. The `LangGraph StateGraph` begins at the `process` node. Inside `process_node()`, the retriever embeds the query using the same `nomic-embed-text` model (ensuring vector space compatibility) and performs MMR search. MMR retrieval uses cosine similarity to find an initial candidate set of `fetch_k=6` chunks, then applies a diversity penalty to select the final `k=3` chunks that are both relevant and maximally different from each other. This prevents the context window from being filled with near-duplicate passages.

The three retrieved chunks are joined into a single context string, which is embedded into a structured prompt. The prompt includes a system instruction ('Answer ONLY using the provided context. If the answer is not in context, say I don't know'), the context block, and the user question. This prompt is sent to `ChatGroq`, which calls the Groq inference API and returns a response. The answer is written into the `GraphState`.

The decision node then calls `route_decision()`, which implements the routing policy described in Section 5. Based on the routing outcome, `LangGraph` follows either the `respond` edge to the output node (which passes the state through) or the `escalate` edge to the escalation node (which replaces the LLM answer with a human-typed response).

3. Design Decisions

3.1 Chunk Size Choice (700 characters, 150 overlap)

A chunk size of 700 characters was chosen as a balance between two competing forces. Smaller chunks improve retrieval precision (the returned chunk closely matches the query) but reduce context density (the answer may require information from multiple chunks that are not all retrieved). Larger chunks provide more context per chunk but reduce retrieval precision and risk exceeding the LLM's effective context window for the relevant information. At approximately 700 characters, a chunk holds 2 to 4 dense sentences of product documentation, which is sufficient to answer most single-fact questions. The 150-character overlap was set at approximately 20% of the chunk size, a common heuristic for minimizing boundary-split information loss.

3.2 Embedding Strategy (nomic-embed-text via Ollama)

The nomic-embed-text model was selected for three reasons: it is free to run locally via Ollama (no API cost for the embedding step), it produces 768-dimensional vectors that are well-calibrated for English retrieval tasks, and it is available without GPU hardware, running acceptably on CPU for development-scale knowledge bases. The decision to use a local embedding model also ensures that the ingestion step does not incur per-token costs.

3.3 Retrieval Approach (MMR over cosine similarity)

Standard cosine similarity retrieval (`search_type='similarity'`) often returns multiple near-duplicate chunks from the same section of the document. For a customer support context where queries are broad (e.g., 'tell me about your return policy'), this produces redundant context. MMR diversifies the retrieved set, ensuring each of the three context chunks contributes unique information to the prompt. The `fetch_k=6` setting gives MMR enough candidates to find diversity without fetching too broadly.

3.4 Prompt Design Logic

The system prompt used in `process_node()` contains three key elements. The role definition ('You are a customer support assistant for NovaTech') anchors the LLM's persona. The grounding constraint ('Answer ONLY using the provided context') directly counters hallucination by making the restriction explicit. The fallback instruction ('If the answer is not in context, say I don't know') ensures that when retrieval fails, the LLM produces an answer that the routing logic can detect and escalate, rather than fabricating a response.

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is used to model the query processing pipeline as a directed graph with explicit state transitions. Unlike a simple function call chain, LangGraph provides two advantages for this system. First, the GraphState object is a typed container that flows through all nodes, making data dependencies explicit and preventing accidental state mutations. Second, the conditional edge mechanism allows routing logic to be cleanly separated from processing logic: the process node focuses purely on retrieval and generation, while the decision node focuses purely on routing policy.

4.2 Node Responsibilities

The process node is responsible for all retrieval and generation work. It is the most computationally intensive node and is the only node that calls external services (the Ollama embedding server and the Groq LLM API).

The decision node is a lightweight routing node. It calls `route_decision()` and writes the string 'respond' or 'escalate' into `state['decision']`. This single key is read by LangGraph's conditional edge lambda to determine the next node.

The respond node (`output_node`) is a no-op pass-through. It receives the final state and returns it unchanged. Its presence as a named node makes the graph structure explicit and allows future functionality (such as logging or response formatting) to be added without modifying the routing logic.

The escalate node calls `human_intervention()` and replaces `state['answer']` with the human-provided response. It preserves the retriever and llm keys in the returned state to maintain a complete state object.

4.3 State Transitions

- Initial state: {query, retriever, llm} - populated by main.py before graph invocation.
- After process node: state gains the 'answer' key containing the LLM response.
- After decision node: state gains the 'decision' key ('respond' or 'escalate').
- After respond node: state is returned as-is; main.py reads `state['answer']`.
- After escalate node: `state['answer']` is overwritten with the human response; main.py reads `state['answer']`.

5. Conditional Logic

5.1 Intent Detection

Intent detection in this system is implemented as rule-based keyword matching rather than a trained classifier. The `route_decision()` function checks two signals: the user's query and the LLM's answer. Query-level detection catches intent before the LLM even processes it, while answer-level detection catches cases where the LLM failed to find an answer regardless of the query's intent.

The keyword list is intentionally minimal in the current implementation: 'refund' and 'payment issue'. Both are high-stakes intents where automated responses carry business and legal risk, making conservative escalation the correct policy. In a production system, this list would be extended and potentially replaced with a small intent classification model (e.g., a fine-tuned BERT classifier) for higher recall and precision.

5.2 Routing Decisions

The routing decision is deterministic given the same query and answer. There are three escalation paths: explicit intent (refund keyword), explicit intent (payment issue keyword), and implicit low confidence (short answer or 'not found' phrase). If none of these are triggered, the system routes to respond. The routing logic does not use probability scores or thresholds; it is a pure rule-based classification.

5.3 Length-Based Confidence Heuristic

The `len(answer) < 30` check is a proxy for answer confidence. If the LLM produces a response shorter than 30 characters, it is almost certainly a non-answer like 'I don't know.' or 'Not found.' rather than a substantive reply. This heuristic catches cases where the LLM follows the grounding instruction and admits ignorance, turning an honest 'I don't know' into a human escalation rather than a dead-end response for the user.

6. HITL Implementation

6.1 Role of Human Intervention

Human-in-the-Loop (HITL) is the mechanism by which the automated system recognizes the limits of its own knowledge or authority and transfers control to a human. In this system, HITL serves two roles. First, it handles high-stakes queries (refunds, payments) where a human must verify eligibility and apply business policy that is not captured in the static knowledge base. Second, it serves as a fallback for low-confidence retrievals, ensuring the user always receives a response rather than a 'I don't know' dead end.

6.2 Implementation Details

The HITL implementation in `hitl/escalation.py` is deliberately simple. The `human_intervention(query)` function prints the escalated query and prompts for a typed response. This simplicity is appropriate for a prototype: it demonstrates the HITL concept without introducing infrastructure dependencies. The function is called from within `escalation_node()` in the LangGraph workflow, which means it runs synchronously within the graph execution. The graph waits at this node until `input()` returns.

6.3 Benefits of HITL

- Prevents incorrect automated responses for sensitive topics (refunds, payments).
- Provides a fallback for any query the RAG pipeline cannot answer confidently.
- Makes the system's limitations transparent and recoverable rather than silently wrong.
- The human response is seamlessly integrated into the same output path as automated responses.

6.4 Limitations of Current Implementation

- Synchronous blocking: The entire application pauses until the human responds.
- No persistence: Escalated queries and human responses are not logged or stored.
- No ticketing integration: A production system would send escalations to a support queue.
- No timeout: If the operator is unavailable, the application hangs indefinitely.

7. Challenges and Trade-offs

Trade-off	Option A	Option B	Decision Made
Retrieval accuracy vs speed	Cosine similarity (faster, simpler)	MMR (slower, more diverse)	MMR chosen. Diversity improves context quality for the LLM at negligible latency cost for small knowledge bases.
Chunk size vs context quality	Small chunks (200-300 chars): high precision	Large chunks (1000+ chars): more context	700 chars chosen as a middle ground providing enough context per chunk without sacrificing retrieval precision.
Cost vs performance (LLM)	GPT-4 (high quality, high cost)	Llama 3.1 8B via Groq (free tier, fast)	Groq Llama chosen. Sufficient instruction-following quality for support use case with no API cost on free tier.
Local vs cloud embeddings	OpenAI text-embedding-3-small (cloud)	nomic-embed-text via Ollama (local)	Local chosen. Eliminates per-token embedding costs and network latency for the ingestion step.
HITL mechanism	Synchronous CLI input()	Async ticket queue	CLI input() chosen for prototype simplicity. Async queue is the production recommendation.

Table 6: System Design Trade-offs

8. Testing Strategy

8.1 Testing Approach

The project includes `main2.py` as a dedicated test runner that validates the pipeline in three progressive stages: retrieval testing, LLM testing, and end-to-end graph testing. This modular approach allows each stage to be debugged in isolation before verifying the integrated system.

8.2 Stage 1 - Retrieval Testing (`test_retrieval`)

`test_retrieval()` runs a fixed query ('What is HTTP?') against the retriever and prints the first 300 characters of each returned chunk. This verifies that ChromaDB is populated correctly, the MMR retriever returns distinct chunks, and the embedding model is functioning as expected.

8.3 Stage 2 - LLM Testing (`test_llm`)

`test_llm()` uses the same 'What is HTTP?' query but manually constructs the context string and calls `llm.invoke()` directly, bypassing the graph. This isolates the LLM response quality from any graph routing issues and verifies that the Groq API key is valid and the ChatGroq instance is correctly configured.

8.4 Stage 3 - Graph Testing (`test_graph`)

`test_graph()` runs the full LangGraph workflow end-to-end and prints the final state['answer']. This validates that all four nodes execute correctly, the conditional edge routes as expected for a routine query, and the graph compiles and runs without errors.

8.5 Sample Queries for Manual Testing

Query	Expected Route	Validation Criteria
What is HTTP?	respond	Answer contains a definition of HTTP sourced from the knowledge base.
I want a refund for my order.	escalate	Escalation triggered immediately on 'refund' keyword; human prompt appears.
I have a payment issue with my account.	escalate	Escalation triggered on 'payment issue' keyword.
What is the meaning of life?	escalate	LLM returns short or 'I don't know' answer; escalated via length check.
How do I reset my password?	respond or escalate	Depends on knowledge base content; if context exists, direct answer is returned.

Table 7: Sample Test Queries

9. Future Enhancements

9.1 Multi-Document Support

The current system is limited to a single `knowledge_base.pdf` file. ChromaDB supports named collections, which would allow multiple documents to be stored in separate namespaces. The retriever could be extended to query across collections or to select the relevant collection based on query intent. A document management layer would handle incremental ingestion (adding new documents without re-embedding the entire knowledge base).

9.2 Feedback Loop

User satisfaction signals (thumbs up/down on answers, follow-up clarification questions) could be logged and used to identify low-quality retrieval cases. Human responses from escalated queries represent a valuable training signal: they are ground-truth answers for questions the RAG system could not handle, and could be used to augment the knowledge base or fine-tune the routing policy.

9.3 Memory Integration

The current system is stateless between turns: each query is processed independently with no memory of prior conversation. LangGraph supports checkpointing via `langgraph-checkpoint`, which would allow conversation history to be persisted and injected into subsequent prompts. This would enable follow-up queries ('Can you elaborate on that?') and context-aware multi-turn conversations.

9.4 Deployment

The CLI interface can be wrapped in a FastAPI application to expose the RAG system as a REST API. A React or Gradio frontend can then provide a chat interface. For cloud deployment, the ChromaDB file store would be replaced with a managed vector database (Pinecone, Weaviate, or Qdrant). The Ollama embedding server would be containerized and deployed alongside the API. The HITL module would be replaced with an integration to a support ticketing system such as Zendesk or Freshdesk, with async webhook-based response handling.

9.5 Evaluation and Observability

A production RAG system requires systematic evaluation of retrieval quality (using metrics such as Mean Reciprocal Rank and NDCG) and generation quality (using RAGAS metrics: faithfulness, answer relevancy, context recall). LangSmith integration (via LangChain's tracing infrastructure) would provide end-to-end observability for every graph execution, capturing retrieved chunks, prompts, LLM responses, and routing decisions in a searchable trace store.

End of Deliverable 3 - Technical Documentation